

Programming Assignment 2 Student Report: Total Order Using Paxos

Cheng, Eric
`eric.cheng2@mail.mcgill.ca`

Zu, Greta
`greta.zu@mail.mcgill.ca`

November 4, 2024

1 Project Description

1.1 Architecture

The general architecture of our Paxos consensus algorithm extends the one shown in class to instill total order while not throwing out competing rounds and handling some failure cases. The implementation uses a majority consensus mechanism (at the proposal stage and the accept phase) to ensure that a single process takes charge of each round and that a single value is committed for each round. This ensures total order since a majority of processes agree on a value and this value is then installed for all.

Since each Paxos instance acts as a Proposer and Acceptor at the same time, we implemented a listenerThread that continuously reads from the GCL using `readGCMessage()`, and based on the message tag received, decides on what to do with the message. For any propose, accept or commit messages, it hands them off to the acceptor methods to handle. For anything else, the message is added to a `readBuffer`, from which the proposer side reads from.

On the proposer side, the `broadcastTOMsg` and `acceptTOMsg` methods are implemented. Each process broadcasts proposals, promises, acceptances, and commits using the GCL (Group Communication Library) instance, which sends messages to all group members. The broadcast method ensures every participant has the chance to contribute and receive updates, ensuring all game states are synchronized. On the other hand, the accept method reads from a `commitQueue` of messages ready to be committed, sorted in total order (ensured by the rest of the Paxos algorithm).

As a rule, we used the `roundNumber` to track where in the game we are, and also as a way to choose the proposer for each turn. The game progresses by increasing the `roundNumber`. To make sure each committed move is unique and not repeated, we used a globally unique index (a combination of the process string - port number - and the round number). *Note: the port number of each process in the group has to be different and unique.

Finally, we also have a shutdown method that interrupts the listener thread and shuts down the GCL to ensure that everything is processed correctly in the case of a graceful shutdown.

1.2 Design

On a high-level, the design of our Paxos consensus algorithm uses a majority voting mechanism to ensure total order of all moves across the boards of the players. Some key designs we used to ensure the above:

- Nested private inner class `PaxosMessage` acting as a message to be passed around. It holds information such as the message type (used for the listenerThread - and decided in the ENUM `PaxosMessageType`), the `roundNumber`, the value, the process and an index (for failure handling in section 1.6).
- As mentioned in section 1.1, we implemented a `startListener()` who starts a listenerThread, constantly reading the GCL for incoming messages and handling them appropriately.

- The broadcastTOMsg method that is called when a process wants to make a move in the game: first, the proposer needs to "win" the ability to propose in the given round. So, it will take the roundNumber and valIndex (unique identifier for each message) and broadcast it's message to all the other processes. During waitForPromises(), it waits until either a majority of processes have accepted it or if its a tie/reject, and returns to the broadcast method. Depending on the situation, if it is not a majority of promises, it will sleep a random amount of time before trying again for the next round. Otherwise, it will continue by sending a message to propose a value. Using the same concept, it will then loop through waiting for acceptances, until it gets the majority or answers (acceptACK or deny). If it is a denial in majority or tie, it will loop back as well to try again the next round. Else, it will broadcast the commit message to everyone. Then, the commit is handled by the acceptor side, added to a queue to wait for the acceptTOMsg call.
- The acceptTOMsg method will then take from the commitQueue if it is not empty, and return the value to the application layer.
- The acceptor side of the implementation consists of the handlePropose(), handleAccept() and handleCommit() methods. They can only vote once for every round, ensuring the election of a single proposer and ensuring the acceptance of a single consensus value for the round. In the case that a commit is sent twice (see section 1.6 for possible failure cases), the handleCommit() will also ensure that the commit is only sent once to the application layer, by checking a committedValues hashmap using the unique index as a key.

1.3 How you ensure that moves made by various players simultaneously eventually make it to all the game instances.

To ensure that the moves made by various players simultaneously eventually make it to all the game instances, and thus also ensuring total order, our broadcastTOMsg method makes sure to never "drop" a move (a move is never lost), and it will send the move to a commitQueue (in order because of the global variables roundNumber and valIndex) from which the next moves are taken from. Throughout, each process shares the move to all other instances, and only with a majority of acceptances can the value be broadcasted to everyone and committed at that point. At the commit stage, all processes must install the value to ensure that they are synced up.

If any instance misses a broadcast (due to network delays or other transient issues), the system retries in the next round, ensuring that missed messages eventually reach all processes. This is done in the handleAccept() method, by having the acceptor accept a proposer but also returning a previously accepted but not yet committed value. Thus initiating a takeover mechanism (also used for handling failures - see section 1.6).

1.4 How you ensure that every process has a fair chance in pushing its move when the rate of moves generated increases (and is not overwhelmed by a process making most of the moves).

To mitigate the possible issues where some processes may dominate with a higher rate of moves, we simply have a single global variable roundNumber that controls the ballotID the proposers can choose from. Then, for the voting rounds, each acceptor can only vote once, to the first proposer message they see. From the perspective of our implementation, each process has a fair and same chance to win each round. It'll simply depend on timing and GCL sending/delivery order. This gives every process a fair opportunity to propose and agree on moves without any single process overwhelming others.

1.5 How you ensure that although your broadcast API only waits for a majority to accept the value, how the rest of the processes still gets the value.

Our implementation does not wait for all the processes to return an accept message, but rather, it only waits for a majority to accept. Once that is done it immediately goes into the commit phase. To ensure that the rest of the processes still get the value, we simply make sure that in the commit phase,

the values are all put into a commitQueue, and the value for the round is then broadcasted to all the processes. At this point, all processes must accept the value no matter what.

1.6 How you handle various failure scenarios (specifically the ones discussed in your project description).

- Failure of the process when it receives a propose message: the Paxos algorithm doesn't behave any differently as long as the majority of processes are still up and running. The way we implemented the algorithm depends on the votes received, so a missing vote from a failed process does not impact the rest of the code. As an extra layer of safety, we also added a timer that would break out of the waiting loop (promises and accepts). In case of less processes, it will simply try again the next round.
- Failure of the process after it sends out its vote (promise/refuse) for a leader election: As long as the majority of processes are still up and running, this should not impact the rest of the algorithm neither. Any situation will be taken care of at the accept phase. If the proposer receives a majority of promises, it will simply move on to the accept phase, and at that point, simply receive one less vote. The timer applies here as well as a precaution in case majority cannot be reached (so that the code won't hang).
- Failure of the process after it sends out its proposal to become a leader: the acceptors will still vote for the proposer (promise or reject) but the sendMsg() will simply not reach the failed process (taken care of in the GCL). The move is lost (which should not matter since the process failed) and the round is skipped. The algorithm will move onto the next move naturally.
- Failure of the process after it sees that it has been accepted by the majority as the leader: if it fails once it has been accepted as the leader for that round, it simply won't be sending out its value to be accepted. The result is the same as the previous case, since nothing will be broadcasted to the acceptors to accept and thus, once the next round comes in, the algorithm simply drops this round and moves on. This move is lost (also does not matter since the process failed).
- Failure of the process once a majority has accepted its proposed value: the process keeps track of any previous value accepted with the corresponding unique index. So if a new proposer comes in for the next round and a majority accepted a value already from the previous round, a takeover mechanism is initiated. The new proposer will have to complete the accepted value from the previous round, and commit it to the queue (as it commits, it also double checks to make sure it wasn't already committed using the index and committedValues - a possible case if the previous process did not fail but rather was just slow). Once the takeover is completed, the proposer will circle back with its own value to fight for a new round.

All in all, our Paxos implementation employs a robust design for distributed consensus, enabling fair participation, ensuring eventual consistency, and handling failure scenarios effectively. By leveraging majority voting, committing broadcasted moves, and using fallback mechanisms, it provides reliable synchronization across game instances, contributing to a resilient distributed system.

2 Collaboration

For this project, since the task division was harder to make, we simply discussed an implementation idea first, meeting up in person to talk through ideas and brainstorm. Once we had a general idea, Eric started the implementation of a general Paxos framework. Greta took over to finish the implementation and add failure handling. Once we had a working structure, we then met up to do some testing, figuring out how to fix the bugs and do failure handling. Finally, Eric did the performance analysis and Greta wrote the report. We then called on discord to go over both reports and review it for mistakes.